NOIP2017 题目选讲及应试策略

wzj52501

Peking University

2018 年 10 月 7 日

NOIP2017 普及组试题选讲

棋盘

有一个 $m \times m$ 的棋盘，棋盘上每一个格子可能是红色、黄色或没有任何颜色的。你现在要从棋盘的最左上角走到棋盘的最右下角。
任何一个时刻，你所站在的位置必须是有颜色的（不能是无色的），你只能向上、下、左、右四个方向前进。当你从一个格子走向另一个格子时，如果两个格子的颜色相同，那你不需要花费金币。如果不同，则你需要花费 1 个金币。
另外，你可以花费 2 个金币施展魔法让下一个无色格子暂时变为你指定的颜色。但这个魔法不能连续使用，而且这个魔法的持续时间很短，也就是说，如果你使用了这个魔法，走到了这个暂时有颜色的格子上，你就不能继续使用魔法；只有当你离开这个位置，走到一个本来就有颜色的格子上的时候，你才能继续使用这个魔法，而当你离开了这个位置（施展魔法使得变为有颜色的格子）时，这个格子恢复为无色。
现在你要从棋盘的最左上角，走到棋盘的最右下角，求花费的最少金币是多少？
$m \leq 100, n \leq 1000$。

cont'd

- 考虑用最短路算法解决，设 $d(i,j,c)$ 表示从 $(1,1)$ 走到 (i,j)，当前格子颜色为 c 所花费的最小金币。

- 考虑用最短路算法解决，设 $d(i,j,c)$ 表示从 $(1,1)$ 走到 (i,j)，当前格子颜色为 c 所花费的最小金币。
- 对于每个状态 (i,j,c)：如果 (i,j) 是无色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边。

cont'd

- 考虑用最短路算法解决，设 $d(i,j,c)$ 表示从 $(1,1)$ 走到 (i,j)，当前格子颜色为 c 所花费的最小金币。
- 对于每个状态 (i,j,c)：如果 (i,j) 是无色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边。
- 如果 (i,j) 是有色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边，并向相邻的无色格子 (i_2,j_2,c_2) 连接权值为 $2+(c_2 \bigoplus c)$ 的边。

cont'd

- 考虑用最短路算法解决，设 $d(i,j,c)$ 表示从 $(1,1)$ 走到 (i,j)，当前格子颜色为 c 所花费的最小金币。
- 对于每个状态 (i,j,c)：如果 (i,j) 是无色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边。
- 如果 (i,j) 是有色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边，并向相邻的无色格子 (i_2,j_2,c_2) 连接权值为 $2+(c_2 \bigoplus c)$ 的边。
- 计算出左上角到右下角的最短路，如果使用堆优化的 Dijkstra 算法时间复杂度为 $O(m^2 \log m)$。

cont'd

- 考虑用最短路算法解决，设 $d(i,j,c)$ 表示从 $(1,1)$ 走到 (i,j)，当前格子颜色为 c 所花费的最小金币。
- 对于每个状态 (i,j,c)：如果 (i,j) 是无色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边。
- 如果 (i,j) 是有色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边，并向相邻的无色格子 (i_2,j_2,c_2) 连接权值为 $2+(c_2 \bigoplus c)$ 的边。
- 计算出左上角到右下角的最短路，如果使用堆优化的 Dijkstra 算法时间复杂度为 $O(m^2 \log m)$。
- 实际上对于权值种类不超过 k 种的最短路，可以在 $O(|E| \times k)$ 的时间复杂度内解决。

cont'd

- 考虑用最短路算法解决，设 $d(i,j,c)$ 表示从 $(1,1)$ 走到 (i,j)，当前格子颜色为 c 所花费的最小金币。
- 对于每个状态 (i,j,c)：如果 (i,j) 是无色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边。
- 如果 (i,j) 是有色格子，则向相邻的有色格子 $(i_2,j_2,co(i_2,j_2))$ 连接权值为 $c \bigoplus co(i_2,j_2)$ 的边，并向相邻的无色格子 (i_2,j_2,c_2) 连接权值为 $2+(c_2 \bigoplus c)$ 的边。
- 计算出左上角到右下角的最短路，如果使用堆优化的 Dijkstra 算法时间复杂度为 $O(m^2 \log m)$。
- 实际上对于权值种类不超过 k 种的最短路，可以在 $O(|E| \times k)$ 的时间复杂度内解决。
- 另外这道题目似乎可以使用记忆化搜索解决，毕竟 NOIP 普及组的搜索真的非常玄学 (

跳房子

跳房子的游戏规则如下：
在数轴上画 n 个格子，第 i 个格子的坐标是 x_i。每个格子内有一个整数 s_i，表示到达这个格子能得到的分数。玩家第一次从起点开始向右跳，跳到起点右侧的一个格子内。第二次再从当前位置继续向右跳，依此类推。规则规定：
玩家每次都必须跳到当前位置右侧的一个格子内。玩家可以在任意时刻结束游戏，获得的分数为曾经到达过的格子中的数字之和。
现在小 R 研发了一款机器人来参加游戏，但是这个机器人每次弹跳的距离只能为固定的 d。小 R 希望改进他的机器人，如果他花 g 个金币改进他的机器人：当 $g < d$ 时，他的机器人每次弹跳的距离在区间 $[d-g, d+g]$ 中；当 $g \geq d$ 时，他的机器人每次弹跳的距离在区间 $[1, d+g]$ 中。
现在小 R 希望获得至少 k 分，请问他至少要花多少金币来改造他的机器人。
$n \leq 5 \times 10^5, 1 \leq d, x_i, k \leq 10^9, |s_i| \leq 10^3$

cont'd

- 首先付出的金币越多得到的分数也会越多，这直接提示了我们可以二分答案。

cont'd

- 首先付出的金币越多得到的分数也会越多，这直接提示了我们可以二分答案。
- 现在的问题转化成了判定一个每次弹跳距离在 $[l, r]$ 的机器人参加一次跳房子游戏的最大得分是否 $\geq k$。

cont'd

- 首先付出的金币越多得到的分数也会越多，这直接提示了我们可以二分答案。
- 现在的问题转化成了判定一个每次弹跳距离在 $[l, r]$ 的机器人参加一次跳房子游戏的最大得分是否 $\geq k$。
- 设 f_i 表示当前机器人跳到了第 i 个格子的最大得分，易得 $f_i = \max\{f_j\} + s_i \quad (x_i - r \leq x_j \leq x_i - l)$

cont'd

- 首先付出的金币越多得到的分数也会越多，这直接提示了我们可以二分答案。
- 现在的问题转化成了判定一个每次弹跳距离在 $[l, r]$ 的机器人参加一次跳房子游戏的最大得分是否 $\geq k$。
- 设 f_i 表示当前机器人跳到了第 i 个格子的最大得分，易得 $f_i = \max\{f_j\} + s_i \quad (x_i - r \leq x_j \leq x_i - l)$
- 随着 i 从小到大递增，可行的 j 区间也向右滑动，对于这种滑动窗口问题，可以使用单调队列优化到 $O(n)$。

- 首先付出的金币越多得到的分数也会越多，这直接提示了我们可以二分答案。
- 现在的问题转化成了判定一个每次弹跳距离在 $[l, r]$ 的机器人参加一次跳房子游戏的最大得分是否 $\geq k$。
- 设 f_i 表示当前机器人跳到了第 i 个格子的最大得分，易得 $f_i = \max\{f_j\} + s_i \quad (x_i - r \leq x_j \leq x_i - l)$
- 随着 i 从小到大递增，可行的 j 区间也向右滑动，对于这种滑动窗口问题，可以使用单调队列优化到 $O(n)$。
- 总时间复杂度为 $O(n \log n)$。

NOIP2017 提高组试题选讲

小凯的疑惑

小凯手中有两种面值的金币，两种面值均为正整数且彼此互素。每种金币小凯都有无数个。在不找零的情况下，仅凭这两种金币，有些物品他是无法准确支付的。现在小凯想知道在无法准确支付的物品中，最贵的价值是多少金币？注意：输入数据保证存在小凯无法准确支付的商品。

$a, b \leq 10^9$。

小凯的疑惑

小凯手中有两种面值的金币，两种面值均为正整数且彼此互素。每种金币小凯都有无数个。在不找零的情况下，仅凭这两种金币，有些物品他是无法准确支付的。现在小凯想知道在无法准确支付的物品中，最贵的价值是多少金币？注意：输入数据保证存在小凯无法准确支付的商品。

$a, b \leq 10^9$。

- 经过一些打表、猜测后，可以知道答案是 $ab - a - b$。

小凯的疑惑

小凯手中有两种面值的金币，两种面值均为正整数且彼此互素。每种金币小凯都有无数个。在不找零的情况下，仅凭这两种金币，有些物品他是无法准确支付的。现在小凯想知道在无法准确支付的物品中，最贵的价值是多少金币？注意：输入数据保证存在小凯无法准确支付的商品。

$a, b \leq 10^9$。

- 经过一些打表、猜测后，可以知道答案是 $ab - a - b$。
- 证明的话可以考虑同余最短路，由于 $(a, b) = 1$ 所以在 $\mod b$ 意义下最大的一个 $dist$ 必然是 $ab - a$，于是最大的不能被表示出的数必然是 $ab - a - b$，证毕。

题目描述

小明正在学习一种新的编程语言 A++，刚学会循环语句的他激动地写了好多程序并 给出了他自己算出的时间复杂度，可他的编程老师实在不想一个一个检查小明的程序， 于是你的机会来啦！下面请你编写程序来判断小明对他的每个程序给出的时间复杂度是否正确。

A++语言的循环结构如下：

```
F i x y
    循环体
E
```

其中 `F i x y` 表示新建变量 i（变量 i 不可与未被销毁的变量重名）并初始化为 x ，然后判断 i 和 y 的大小关系，若 i 小于等于 y 则进入循环，否则不进入。每次循环结束后 i 都会被修改成 $i+1$ ，一旦 i 大于 y 终止循环。

x 和 y 可以是正整数（ x 和 y 的大小关系不定）或变量 n 。 n 是一个表示数据规模的变量，在时间复杂度计算中需保留该变量而不能将其视为常数，该数远大于 100 。

“E”表示循环体结束。循环体结束时，这个循环体新建的变量也被销毁。

注：本题中为了书写方便，在描述复杂度时，使用大写英文字母“O”表示通常意义下“Θ”的概念。

程序语法可能有错误，程序行数不超过 100。

cont'd

- 首先是如何判断程序语法是否出错的问题，这相当于判定一个括号序列是否完全匹配和判断未销毁变量名是否重复，后者只需要维护一个栈就可以了。

- 首先是如何判断程序语法是否出错的问题，这相当于判定一个括号序列是否完全匹配和判断未销毁变量名是否重复，后者只需要维护一个栈就可以了。
- 现在我们得到了一个语法无误的程序，共有 5 种情况：
  1. F i 常量 a 常量 $b(a \leq b)$：将其视为权值为 0 的左括号。
  2. F i 常量 a 常量 $b(a > b)$：将其视为权值为 $-inf$ 的左括号。
  3. F i 常量 a n：将其视为权值为 1 的左括号。
  4. F i n 常量 b：将其视为权值为 $-inf$ 的左括号。
  5. F i n n：将其视为权值为 0 的左括号。

cont'd

- 首先是如何判断程序语法是否出错的问题，这相当于判定一个括号序列是否完全匹配和判断未销毁变量名是否重复，后者只需要维护一个栈就可以了。
- 现在我们得到了一个语法无误的程序，共有 5 种情况：
  1. F i 常量 a 常量 $b(a \leq b)$：将其视为权值为 0 的左括号。
  2. F i 常量 a 常量 $b(a > b)$：将其视为权值为 $-inf$ 的左括号。
  3. F i 常量 a n：将其视为权值为 1 的左括号。
  4. F i n 常量 b：将其视为权值为 $-inf$ 的左括号。
  5. F i n n：将其视为权值为 0 的左括号。
- 程序的时间复杂度即为括号的前缀和的最大值。

奶酪

现有一块大奶酪，它的高度为 h，它的长度和宽度我们可以认为是无限大的，奶酪中间有许多半径均为 r 的球形空洞。我们可以在这块奶酪中建立空间坐标系，在坐标系中，奶酪的下表面为 $z = 0$，奶酪的上表面为 $z = h$。

现在，奶酪的下表面有一只小老鼠 Jerry，它知道奶酪中所有空洞的球心所在的坐标。如果两个空洞相切或是相交，则 Jerry 可以从其中一个空洞跑到另一个空洞，特别地，如果一个空洞与下表面相切或是相交，Jerry 则可以从奶酪下表面跑进空洞；如果一个空洞与上表面相切或是相交，Jerry 则可以从空洞跑到奶酪上表面。

位于奶酪下表面的 Jerry 想知道，在不破坏奶酪 的情况下，能否利用已有的空洞跑到奶酪的上表面去?

空间内两点 $P_1(x_1, y_1, z_1)$、$P2(x_2, y_2, z_2)$ 的距离公式为 $dist = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$。

$n \leq 1000, 1 \leq h, r \leq 10^9$，坐标绝对值不超过 10^9。

- 将每个空洞和上下表面分别看成一个点，将可以互相到达的点连上边，接下来只需要用并查集或者 BFS 判断两点是否连通即可。

cont'd

- 将每个空洞和上下表面分别看成一个点，将可以互相到达的点连上边，接下来只需要用并查集或者 BFS 判断两点是否连通即可。
- 判断 $dist \leq 2r$ 时可以用 $dist^2 \leq 4r^2$，可以避免实数误差和减小常数。

cont'd

- 将每个空洞和上下表面分别看成一个点，将可以互相到达的点连上边，接下来只需要用并查集或者 BFS 判断两点是否连通即可。
- 判断 $dist \leq 2r$ 时可以用 $dist^2 \leq 4r^2$，可以避免实数误差和减小常数。
- 时间复杂度为 $O(n^2)$。

逛公园

策策同学特别喜欢逛公园。公园可以看成一张 n 个点 m 条边构成的有向图，且没有 自环和重边。其中 1 号点是公园的入口，n 号点是公园的出口，每条边有一个非负权值，代表策策经过这条边所要花的时间。
策策每天都会去逛公园，他总是从 1 号点进去，从 n 号点出来。
策策喜欢新鲜的事物，他不希望有两天逛公园的路线完全一样，同时策策还是一个特别热爱学习的好孩子，他不希望每天在逛公园这件事上花费太多的时间。如果 1 号点到 n 号点的最短路长为 d，那么策策只会喜欢长度不超过 $d+k$ 的路线。
策策同学想知道总共有多少条满足条件的路线，你能帮帮他吗？为避免输出过大，答案对 p 取模。
如果有无穷多条合法的路线，请输出 -1。
$n \leq 100000, m \leq 200000, k \leq 50, p \leq 10^9$
$1 \leq u_i, v_i \leq n, 0 \leq c_i \leq 1000$

DP 前的分析

- 首先使用 Dijkstra 算法对所有 $1 \leq i \leq n$ 预处理出 $dist(1, i)$ 和 $dist(i, n)$，后者可以通过反向添边求出。

DP 前的分析

- 首先使用 Dijkstra 算法对所有 $1 \leq i \leq n$ 预处理出 $dist(1, i)$ 和 $dist(i, n)$，后者可以通过反向添边求出。
- 一条从 u 到 v 的长度为 len 的有向边可能出现在答案的路径中，当且仅当 $dist(1, u) + len + dist(v, n) \leq dist(1, n) + k$。把没用的边删去后，每条边都有可能出现在答案中。

DP 前的分析

- 首先使用 Dijkstra 算法对所有 $1 \leq i \leq n$ 预处理出 $dist(1, i)$ 和 $dist(i, n)$，后者可以通过反向添边求出。
- 一条从 u 到 v 的长度为 len 的有向边可能出现在答案的路径中，当且仅当 $dist(1, u) + len + dist(v, n) \leq dist(1, n) + k$。把没用的边删去后，每条边都有可能出现在答案中。
- 将每条从 u 到 v 的长为 len 的有向边的长度替换成 $dist(1, u) - dist(1, v) + len$ 后，问题转化成求从 1 到 n 的，长度 $\leq k$ 的路径的条数。

DP 前的分析

- 首先使用 Dijkstra 算法对所有 $1 \leq i \leq n$ 预处理出 $dist(1, i)$ 和 $dist(i, n)$，后者可以通过反向添边求出。
- 一条从 u 到 v 的长度为 len 的有向边可能出现在答案的路径中，当且仅当 $dist(1, u) + len + dist(v, n) \leq dist(1, n) + k$。把没用的边删去后，每条边都有可能出现在答案中。
- 将每条从 u 到 v 的长为 len 的有向边的长度替换成 $dist(1, u) - dist(1, v) + len$ 后，问题转化成求从 1 到 n 的，长度 $\leq k$ 的路径的条数。
- 以下提到的边均为替换边权后的结果。

cont'd

- 因为把没用的边删去后，每条边都有可能出现在答案中，所以如果此时仍然有仅由长度为 0 的边构成的环，答案一定是无穷大。这可以通过拓扑排序线性判断。

cont'd

- 因为把没用的边删去后，每条边都有可能出现在答案中，所以如果此时仍然有仅由长度为 0 的边构成的环，答案一定是无穷大。这可以通过拓扑排序线性判断。
- 现在考虑没有仅由长度为 0 的边构成的环的情况，设 $f(i,j)$ 表示从 1 到 i 且长度为 j 的路径的数量。

cont'd

- 因为把没用的边删去后，每条边都有可能出现在答案中，所以如果此时仍然有仅由长度为 0 的边构成的环，答案一定是无穷大。这可以通过拓扑排序线性判断。
- 现在考虑没有仅由长度为 0 的边构成的环的情况，设 $f(i,j)$ 表示从 1 到 i 且长度为 j 的路径的数量。
- 考虑递推转移，枚举下一步走了那条边：
- $f(v,j+len)+=f(i,j)$，(存在一条从 i 到 v 且长度为 len 的边)

cont'd

- 因为把没用的边删去后，每条边都有可能出现在答案中，所以如果此时仍然有仅由长度为 0 的边构成的环，答案一定是无穷大。这可以通过拓扑排序线性判断。
- 现在考虑没有仅由长度为 0 的边构成的环的情况，设 $f(i,j)$ 表示从 1 到 i 且长度为 j 的路径的数量。
- 考虑递推转移，枚举下一步走了那条边：
- $f(v,j+len)+=f(i,j)$，(存在一条从 i 到 v 且长度为 len 的边)
- 总时间复杂度为 $O((n+m)\times(k+\log n))$。

杂题选讲

Tree

有一个 n 个节点的二叉树，要求每个节点的两个儿子的子树最大深度相差不超过 1，求最大深度。

$n \leq 10^{18}$

Tree

有一个 n 个节点的二叉树，要求每个节点的两个儿子的子树最大深度相差不超过 1，求最大深度。
$n \leq 10^{18}$

- 设 f_n 为深度为 n 的子树最少需要多少点，则 $f_n = f_{n-1} + f_{n-2} + 1$。

Tree

有一个 n 个节点的二叉树，要求每个节点的两个儿子的子树最大深度相差不超过 1，求最大深度。
$n \leq 10^{18}$

- 设 f_n 为深度为 n 的子树最少需要多少点，则 $f_n = f_{n-1} + f_{n-2} + 1$。
- 类似 Fibonacci 数列，答案在 $O(\log n)$ 级别。

String

给出一个字符串 s，求其最长的子序列 t，使得 t 可以被表示成 ww 的形式（例如 abcabc）

$n \leq 50$

String

给出一个字符串 s，求其最长的子序列 t，使得 t 可以被表示成 ww 的形式（例如 abcabc）
$n \leq 50$

- 枚举分界点做 LCS 算法。

String

给出一个字符串 s，求其最长的子序列 t，使得 t 可以被表示成 ww 的形式（例如 abcabc）
$n \leq 50$

- 枚举分界点做 LCS 算法。
- 时间复杂度为 $O(n^3)$。

Interval

一个长度为 n 的序列 A_i，找出一段区间 $[l, r]$。
这个特殊区间满足，存在一个 $k \in [l, r]$，并且对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。这样的一个特殊区间 $[l, r]$ 价值为 $r - l$。
求知道序列中所有特殊区间的最大价值是多少与方案数。
$n \leq 5 \times 10^5, 1 \leq A_i \leq 10^9$

Interval

一个长度为 n 的序列 A_i，找出一段区间 $[l, r]$。
这个特殊区间满足，存在一个 $k \in [l, r]$，并且对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。这样的一个特殊区间 $[l, r]$ 价值为 $r - l$。
求知道序列中所有特殊区间的最大价值是多少与方案数。
$n \leq 5 \times 10^5, 1 \leq A_i \leq 10^9$

- 枚举 k，则问题转化成找最大的区间 $[l, r](l \leq k \leq r)$ 满足对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。

Interval

一个长度为 n 的序列 A_i，找出一段区间 $[l, r]$。
这个特殊区间满足，存在一个 $k \in [l, r]$，并且对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。这样的一个特殊区间 $[l, r]$ 价值为 $r - l$。
求知道序列中所有特殊区间的最大价值是多少与方案数。
$n \leq 5 \times 10^5, 1 \leq A_i \leq 10^9$

- 枚举 k，则问题转化成找最大的区间 $[l, r](l \leq k \leq r)$ 满足对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。
- 使用 ST 表快速计算区间 gcd，就可以用二分答案的方法 $O(\log \log A_i)$ 计算最大的区间。

Interval

一个长度为 n 的序列 A_i，找出一段区间 $[l, r]$。
这个特殊区间满足，存在一个 $k \in [l, r]$，并且对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。这样的一个特殊区间 $[l, r]$ 价值为 $r - l$。
求知道序列中所有特殊区间的最大价值是多少与方案数。
$n \leq 5 \times 10^5, 1 \leq A_i \leq 10^9$

- 枚举 k，则问题转化成找最大的区间 $[l, r](l \leq k \leq r)$ 满足对于任意的 $i \in [l, r]$，A_i 都能被 A_k 整除。
- 使用 ST 表快速计算区间 gcd，就可以用二分答案的方法 $O(\log \log A_i)$ 计算最大的区间。
- 时间复杂度为 $O(n \log n \log A_i)$。

password

给定一个长度为 n 的小写字母串，你可以交换 m 次其中相邻两位字符，求最后能得到的最小字典序的字符串。

$n \leq 10^5$ $m \leq 10^{18}$

password

给定一个长度为 n 的小写字母串，你可以交换 m 次其中相邻两位字符，求最后能得到的最小字典序的字符串。
$n \leq 10^5$ $m \leq 10^{18}$

- 按位贪心，考虑将最近的字符 c 换到该位置是否可行。

password

给定一个长度为 n 的小写字母串，你可以交换 m 次其中相邻两位字符，求最后能得到的最小字典序的字符串。
$n \leq 10^5$ $m \leq 10^{18}$

- 按位贪心，考虑将最近的字符 c 换到该位置是否可行。
- 注意每次移动 $[l, r]$ 区间的同时会让 $[l, r]$ 中的所有字符整体平移，可以用树状数组来维护每个字符到当前要确定的位置的距离。

password

给定一个长度为 n 的小写字母串，你可以交换 m 次其中相邻两位字符，求最后能得到的最小字典序的字符串。
$n \leq 10^5$ $m \leq 10^{18}$

- 按位贪心，考虑将最近的字符 c 换到该位置是否可行。
- 注意每次移动 $[l, r]$ 区间的同时会让 $[l, r]$ 中的所有字符整体平移，可以用树状数组来维护每个字符到当前要确定的位置的距离。
- 时间复杂度为 $O(n \log n)$。

NOIP 考场小贴士

Tips

Tips

- 任何一道题都不应该爆零！

Tips

- 任何一道题都不应该爆零！
- 不一定要会做，但一定要注重部分分。

Tips

- 任何一道题都不应该爆零！
- 不一定要会做，但一定要注重部分分。
- 考场程序调不出来时可以去洗手间洗洗脸（

Tips

- 任何一道题都不应该爆零！
- 不一定要会做，但一定要注重部分分。
- 考场程序调不出来时可以去洗手间洗洗脸（
- 最后十分钟建议不要再写程序了（当然你自信最后大力出奇迹也是可以的），先检查文件名是否有误，再检查调试信息是否全部删除，然后检查是否可能在运算过程中数组越界或整数溢出，最后检查内存是否超限。

课后练习

- [NOIP2016] 换教室 传送门
- [NOIP2016] 组合数问题 传送门
- [NOIP2016] 蚯蚓 传送门
- [NOIP2016] 愤怒的小鸟 传送门

THX

祝愿大家在 NOIP2018 中取得好成绩！